TP Algorithmique n°6

Les listes en Python

Objectifs de la séance. Ranger plusieurs valeurs dans une seule variable, une **liste** ; accéder à un élément par son **indice** ; ajouter des éléments ; parcourir une liste avec une boucle pour calculer une moyenne, compter, filtrer ou chercher un maximum.

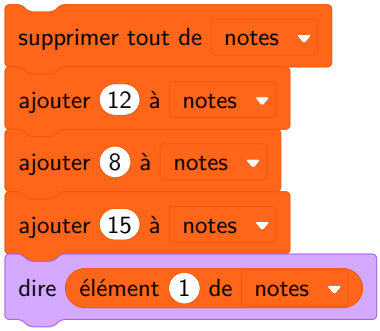
Démarche. L'étiquette de chaque exercice indique l'étape de travail : **Prévoir** (ce que va faire le programme, sans ordinateur), **Exécuter** (vérifier sa prévision), **Comprendre** (expliquer pourquoi le programme marche), **Modifier** (adapter un programme donné), **Créer** (écrire son propre programme).

Ouvrir Capytale depuis l'ENT et saisir le code de l'activité : .....

Partie 1 — Cours

Cours 1 : créer une liste et lire ses éléments

En Scratch



En Python

Script

```
1 notes = [12, 8, 15]
2 print(notes[0])
3 print(len(notes))
```

Console

```
>>> # script executed
12
3
```

À retenir :

- Une liste s'écrit entre **crochets**, les éléments séparés par des virgules : `notes = [12, 8, 15]`.
- Chaque élément a une position, son **indice**. **Attention : en Python, on commence à compter à 0** : `notes[0]` vaut 12, `notes[1]` vaut 8, `notes[2]` vaut 15 (en Scratch, on commence à 1).
- `notes[-1]` est le **dernier** élément ; `len(notes)` est le nombre d'éléments (la **longueur**).
- On modifie un élément par affectation : `notes[1] = 9` remplace le 8 par un 9.

Cours 2 : ajouter des éléments et parcourir une liste

Script

```
1 carres = []
2 for i in range(5):
3     carres.append(i ** 2)
4 print(carres)
```

Console

```
>>> # script executed
[0, 1, 4, 9, 16]
```

Script

```
1 notes = [12, 8, 15]
2 for note in notes:
3     print(note + 1)
```

Console

```
>>> # script executed
13
9
16
```

À retenir :

- `[]` est la liste **vide** ; `L.append(x)` ajoute `x` **à la fin** de la liste `L`.
- `for x in L:` fait prendre à `x` **successivement chaque élément** de la liste (on ne s'occupe pas des indices).
- Si on a besoin des indices : `for i in range(len(L)):` et on utilise `L[i]`.

Fonctions toutes prêtes. `sum(L)` (somme), `max(L)`, `min(L)`, `sorted(L)` (liste triée dans l'ordre croissant), et le test `x in L` (vrai si `x` est dans la liste). Dans les exercices, on les utilisera pour **vérifier** nos propres programmes.

Partie 2 — Exercices

Exercice 1 ★☆☆

Prévoir puis Exécuter

Prévoir, puis vérifier. On a tapé `notes = [12, 8, 15, 10, 17]`. Prévoir ce qu'affiche chaque instruction, puis vérifier dans la console.

On tape	`notes[0]`	`notes[2]`	`notes[-1]`	`len(notes)`	`notes[5]`
Je prévois	.....	.....	.....	.....	.....

Exercice 2 ★☆☆

Modifier

Modifier une liste. On repart de `notes = [12, 8, 15, 10, 17]`.

① Un élève absent rattrape le devoir et a 14 : ajouter cette note à la fin de la liste.

② La deuxième note était fausse : c'est 9 et non 8. La corriger.

③ Afficher la liste. On obtient ..... et elle contient ..... notes.

.....

.....

.....

Exercice 3 ★★☆

Modifier puis Créer

Calculer une moyenne. Voici les notes de 15 élèves (exemple du cours de statistiques) :

`notes = [6, 8, 9, 9, 10, 11, 11, 12, 12, 12, 13, 14, 15, 15, 17]`

① Compléter la fonction `moyenne(L)`, qui calcule la somme des éléments avec une boucle (sans utiliser `sum`), puis divise par le nombre d'éléments.

Script

```
def moyenne(L):
    total = 0
    for x in L:
        total = ..............
    return total / ...........
```

② `moyenne(notes)` renvoie ..... Vérifier avec `sum(notes) / len(notes)`.

③ Le professeur ajoute 2 points à tout le monde. Écrire un script qui construit la liste `nouvelles` des nouvelles notes, puis calculer sa moyenne. Résultat : ..... Était-ce prévisible ?

.....

.....

.....

.....

Exercice 4 ★★☆

Créer

Construire des listes avec une boucle.

① Construire la liste des carrés des entiers de 0 à 10. Son dernier élément est `.....`.

② Construire la liste des multiples de 7 compris entre 1 et 100 (on utilisera `%`). Elle contient `.....` nombres.

③ **Arithmétique.** Écrire une fonction `diviseurs(n)` qui renvoie la liste des diviseurs positifs de `n`.
`diviseurs(36)` renvoie `.....`

Exercice 5 ★★☆

Créer

Compter et filtrer. On reprend la liste `notes` des 15 notes de l'exercice 3.

① Écrire une fonction `compter_moyenne(L)` qui renvoie le nombre de notes supérieures ou égales à 10. Résultat : `.....`.

② Écrire une fonction `admis(L)` qui renvoie la **liste** de ces notes.

Exercice 6 ★★★

Comprendre & Modifier

Chercher le maximum sans `max`.

① Compléter la fonction `maximum(L)`. Idée : on garde dans `m` le plus grand élément vu jusqu'ici.

Script

```
def maximum(L):
    m = L[0]
    for x in L:
        if ............:
            m = ......
    return m
```

② Pourquoi initialise-t-on `m` avec `L[0]` et pas avec `0` ? Penser à la liste `[-5, -2, -8]`.

③ *Bonus.* Modifier la fonction pour qu'elle renvoie aussi l'**indice** du maximum. Pour la liste `notes`, le maximum

17 est à l'indice .....

Exercice 7 ★★★

Créer

Pour aller plus loin : simuler des lancers de dé. L'instruction `randint(1, 6)` (après `from random import randint`) donne un entier **au hasard** entre 1 et 6.

① Construire une liste `lancers` de 20 résultats de lancers de dé.

② Compter le nombre de 6, puis calculer la fréquence d'apparition du 6.

③ Recommencer avec 10 000 lancers. De quel nombre la fréquence se rapproche-t-elle ? .....